

CrashGuard: Handlebar-Mounted Crash Detection and Emergency Alert System

Design, Simulation, and Evaluation

Adheesh Garg (23BCT0122)

Aastha Gupta (23BCT0082)

Ayan Gatani (23BCT0059)

Programming for IoT Boards Lab (BCSE312P)

Class VL2026270103137 Faculty: Syamasudha Veeragandham (22727)
Assessment 5, Review 2 28 August 2026

Abstract

CrashGuard is a low-cost prototype for detecting fall-like motorcycle events and starting a rider-cancellable emergency alert. It combines an ESP32, handlebar-mounted MPU-6050, LED, piezo alarm, cancel button, and optional Blynk notification. Physical hardware was unavailable for this review, so the evidence is limited to deterministic data replay, an independent powered-two-wheeler dataset audit, native tests, target compilation, and three Wokwi virtual-hardware scenarios. The CrashGuard state machine correlates acceleration, rotation, orientation change, and post-event rest instead of using a single impact threshold. On 675 generated ten-second events it produced 114 true positives, 525 true negatives, 0 false positives, and 36 false negatives, corresponding to 76.0% sensitivity and 100.0% specificity. These are simulation results and must not be interpreted as road-crash accuracy. The study also shows that the original 4 g and 200 °/s rule is too strict for the closest public motorcycle data. The outcome is a testable prototype and a clear plan for later bench and road validation.

Evidence statement. No physical crash, battery-runtime test, or network-delivery measurement is claimed. Wokwi verifies virtual sensor, firmware, button, and GPIO behavior; it does not reproduce vehicle mechanics or prove field safety.

Contents

1	Introduction and scope	3
2	Evidence from the base paper and related work	3
3	System design	4
4	Data and decision method	7
5	Implementation and Wokwi validation	8
6	Results and interpretation	9
7	Limitations and safety	11
8	Conclusion	11

1 Introduction and scope

India recorded 172,890 road deaths in 2023, including 77,539 two-wheeler users [1]. CrashGuard addresses only the delay between a fall-like inertial event and an alert attempt. It does not prevent crashes, estimate injury, guarantee Internet access, or replace emergency services.

The project investigates three questions:

RQ1: Can impact, rotation, and tilted rest reject high-dynamic non-crash riding events better than acceleration alone?

RQ2: Is a strict 4 g and 200 °/s ordered rule consistent with independent powered-two-wheeler recordings?

RQ3: Can one compact decision method run in native replay and on an ESP32 while producing the expected virtual-hardware behavior?

The main contribution is an interpretable state machine with a complete electrical design, repeatable simulated event set, public-data sanity check, and automated Wokwi scenarios. Machine learning is deferred because no large, representative, labelled dataset exists for this exact vehicle and mounting position.

2 Evidence from the base paper and related work

The supplied pIOT base paper uses a shin-mounted LSM6DSM, 50 Hz windows, and a CNN-transformer for human fall recognition [2]. CrashGuard adopts its useful principles—six-axis temporal context, separate sensitivity and specificity, and hardware-aware design—but not its model or score. Table 1 makes the difference explicit.

The paper was treated as evidence to audit, not final truth. Its stated ROC point (FPR, TPR) = (0.055, 0.905) implies an AUC of at least $0.905(1 - 0.055) = 0.855$ for any valid nondecreasing ROC curve, which conflicts with the reported AUC of 0.768. It also describes the fixed-point format as both 8-bit and Q7.8, and the downloaded archive does not contain a complete manifest for reproducing the paper’s final windows, labels, augmentation, and subject splits. Therefore, neither its numerical result nor model is reused as CrashGuard performance evidence.

Table 1: Comparison with the supplied base paper. Results are not a head-to-head benchmark.

Item	Base paper	CrashGuard
Target and sensor	Human falls; shin LSM6DSM, 50 Hz	Motorcycle fall-like events; handlebar MPU-6050, 100 Hz
Decision method	Balanced CNN-transformer with focal loss	Fixed, interpretable temporal state machine
Reported evidence	Test sensitivity 90.5%, specificity 94.5%, accuracy 93.1% on its subject-separated human data	Simulated sensitivity 76.0% and specificity 100.0% on 675 generated events
Deployment	Reported iCE40UP5K FPGA implementation	ESP32 target compile, native replay, and Wokwi virtual hardware

The base-paper percentages must not be compared numerically with CrashGuard’s: the sensing body, label definition, dataset, split, and evaluation unit all differ, and CrashGuard has no physical target-domain test set.

Motorcycle-specific research provides the stronger design basis. Boubezoul et al. published a rule-based powered-two-wheeler detector and released four fall plus seven extreme-maneuvre recordings [3, 4]. Gelmini et al. designed and experimentally validated two-wheeler crash logic [5, 6]. Mahariba et al. applied conventional machine learning to the Boubezoul data [7], while Lee et al. compared threshold, supervised, and unsupervised e-scooter methods [8]. These studies support multi-feature temporal reasoning, but their results are not directly comparable because placements, vehicles, labels, and test protocols differ.

3 System design

The MPU-6050 is sampled at 100 Hz. Detection remains local on the ESP32. A confirmed event activates the LED and piezo, then starts a 15 s cancellation window. If the rider does not cancel, the firmware makes a bounded SSL connection through a phone hotspot and calls Blynk IoT’s `Blynk.logEvent` with event code `crash_confirmed` [9]. The event must be created in the Blynk template with timeline and notification delivery enabled. The hotspot supplies connectivity only; it does not give the ESP32 the phone’s location. No coordinates are therefore invented.

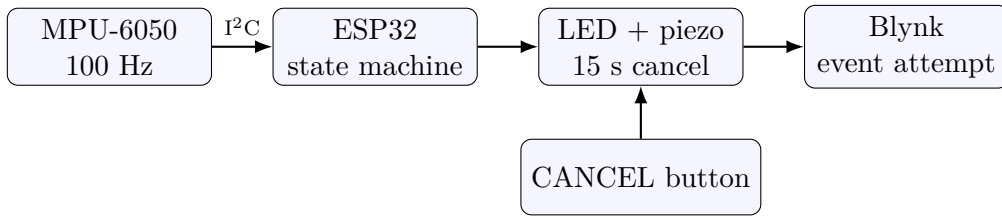


Figure 1: CrashGuard operating sequence. Detection and cancellation do not depend on the network.

Table 2: Main hardware and ESP32 connections.

Function	Connection	Design reason
MPU-6050	SDA GPIO21, SCL GPIO22, 3.3 V	400 kHz I ² C; 3.3 V avoids unsafe 5 V pull-ups
Visual alarm	GPIO26 through 220 Ω LED resistor	Local indication
Piezo alarm	GPIO25 through 1 k Ω and 2N2222	Keeps sounder current out of the GPIO
Cancel button	GPIO27 to ground, internal pull-up	Active-low input without a floating state
Battery monitor	GPIO34 through 100 k Ω /100 k Ω divider	ADC1 remains usable with Wi-Fi
Power	Protected 18650, TP4056/DW01A, MT3608 at 5.0 V	Portable supply with cell protection

The MPU-6050 uses $\pm 16g$ and $\pm 2000^\circ/s$ ranges to reduce immediate saturation during

rapid motion [10]. GPIO34 is input-only and belongs to ADC1, making it suitable for battery sensing while Wi-Fi is active [11]. A 100 μF rail capacitor and local 100 nF capacitors reduce short supply disturbances. The selected TP4056 module is a charger with protection, not a power-path controller; the prototype must be switched off while charging.

At an estimated average 5 V load of 120 mA, the rail power is 0.60 W. With a 3.7 V cell and assumed 90% boost efficiency,

$$I_{cell} \approx \frac{0.60}{3.7(0.90)} = 0.180 \text{ A.} \quad (1)$$

Using 90% of a 2600 mAh cell gives an ideal estimate of about 13 h. This is a sizing calculation, not a measured runtime.

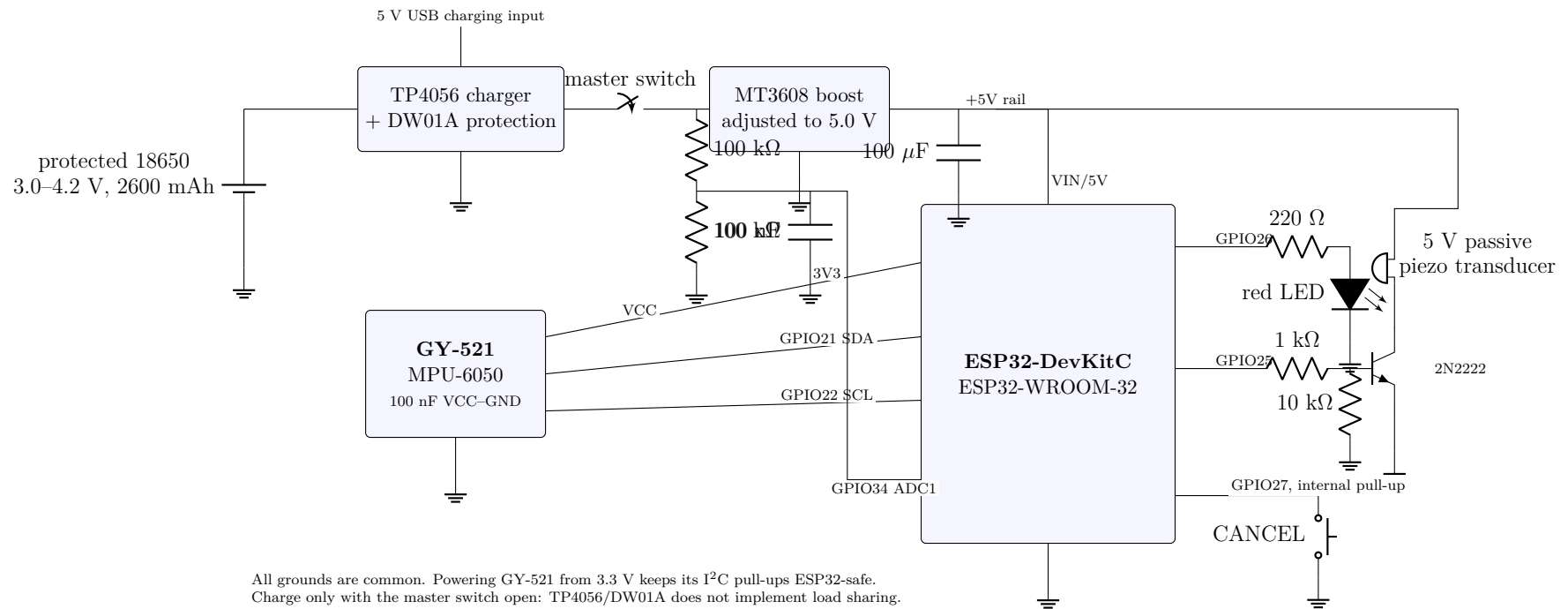


Figure 2: Complete physical circuit. Wokwi uses a low-current virtual piezo directly on GPIO25; physical hardware uses the shown transistor driver.

4 Data and decision method

Three data sources have different roles, summarized in Table 3. The HealthLINK archive was downloaded to audit the base-paper provenance but was excluded from motorcycle metrics because its labels and sensing body do not match this problem [12]. The Boubezoul powered- two-wheeler recordings provide an independent feature check. The generated set provides repeatable complete-event replay.

Table 3: Data used in the study.

Source	Content	Use
HealthLINK	41 human IMU recordings	Base-paper archive audit only
Public PTW dataset	Four falls and seven critical manoeuvres at 1 kHz	Independent impact and rotation sanity check
CrashGuard generator	675 events, 675,000 six-axis samples at 100 Hz	Exact event-level replay of all three decision methods

The generated set uses seed 20260828 and nine groups of 75 events: normal riding, pothole, speed breaker, hard brake, sharp turn, parked tip-over, impact with recovery, low-side crash, and high-side crash. Only the last two are positive. Attitude trajectories produce body-frame gravity; linear pulses model road and impact effects; angular rates come from attitude derivatives. Bias and sample noise are added before clipping to the selected sensor ranges. This improves realism over arbitrary threshold crossings, but it remains a model developed alongside the detector.

For acceleration $\mathbf{a}_k$ and angular rate $\boldsymbol{\omega}_k$ at sample k , the main features are

$$A_k = \|\mathbf{a}_k\|, \quad \Omega_k = \|\boldsymbol{\omega}_k\|, \quad (2)$$

and tilt relative to the stationary startup gravity reference $\mathbf{r}$ is

$$\Theta_k = \cos^{-1} \left(\text{clip} \frac{\mathbf{a}_k \cdot \mathbf{r}}{\|\mathbf{a}_k\| \|\mathbf{r}\|}, -1, 1 \right). \quad (3)$$

CrashGuard first requires recent ride-like activity. It then opens a candidate when acceleration is at least 2.0 g or at most 0.35 g. Rotation of at least 90 °/s may occur up to 750 ms before or after this event. The candidate is confirmed only if tilt reaches 55° while acceleration remains between 0.70 and 1.30 g and angular rate stays below 30 °/s continuously for 2 s. Failure to establish rest within 6 s rejects the event. These rules are summarized in Table 4.

Table 4: CrashGuard decision parameters.

Quantity	Value	Purpose
Acceleration candidate	$A \geq 2.0g$ or $A \leq 0.35g$	Impact or unloading
Rotation	$\Omega \geq 90^\circ/\text{s}$	Fall-like angular motion
Correlation interval	750 ms	Allows either temporal order
Tilt	$\Theta \geq 55^\circ$	Vehicle no longer upright
Rest	$0.70 \leq A \leq 1.30g$, $\Omega \leq 30^\circ/\text{s}$	Stable final pose
Rest duration	2000 ms	Rejects short disturbances
Cancel interval	15000 ms	Gives the rider time to stop alerting

The comparison uses: (1) acceleration at or above 2 g alone, (2) a strict ordered rule requiring 4 g, then 200 °/s, then three seconds of rest, and (3) CrashGuard. The first two are comparison methods, not older releases of CrashGuard. No method is trained or fitted on the event labels.

5 Implementation and Wokwi validation

The production firmware directly configures and reads the MPU-6050, samples at 100 Hz, calibrates bias and upright gravity, applies the state machine, handles the active-low cancel button, and drives the alarms. The same portable C++ decision logic is used for native dataset replay, avoiding disagreement between the reported algorithm and deployed code. Native tests cover rotation before impact, pothole rejection, parked tip rejection, and cancellation.

Compilation with ESP32 Arduino core 3.3.11 and Blynk library 1.3.5 succeeds, using 1,040,343 bytes of flash (79%) and 49,124 bytes of static RAM (14%). The firmware starts Wi-Fi without blocking detection and connects to Blynk only when the cancellation deadline expires. A submitted event is distinguished from an unavailable network and from verified phone receipt. Build size shows feasibility only; timing jitter, energy use, and long-duration stability still require physical measurement.

The Wokwi circuit contains an ESP32-DevKitC virtual board, MPU-6050, LED, piezo, button, and battery-sense potentiometer. It uses the same GPIO numbers as the physical design. UART is connected explicitly so the test runner can observe firmware events. Wokwi sensor controls apply values in g and degree/s [13–15].

The virtual crash, pothole, and cancellation scenarios were executed once and passed with Wokwi CLI 0.26.1. The checked-in offline build contains no Blynk credentials and explicitly reports that the event was skipped. A separate opt-in scenario expects BLYNK_EVENT_SUBMITTED when a dedicated Blynk test device is configured over Wokwi’s Wokwi-GUEST network; it is not reported as a completed phone-delivery test.

Table 5: Wokwi virtual-hardware results.

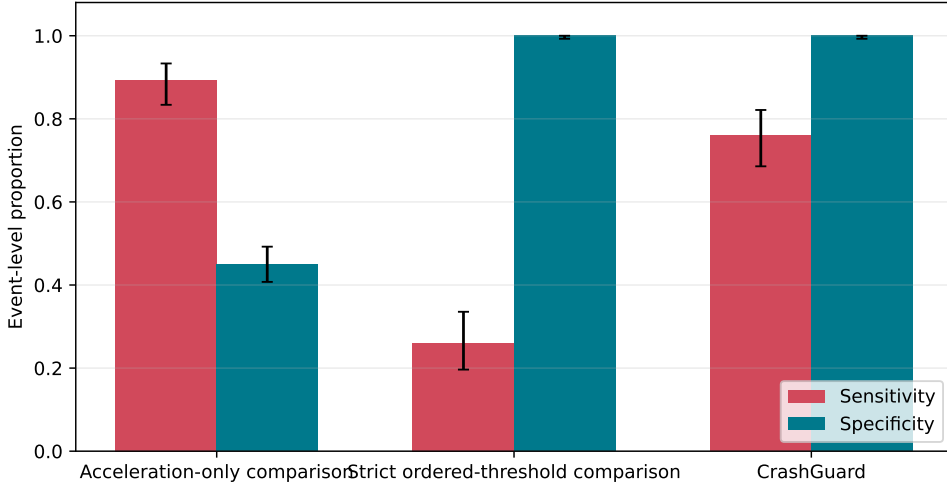
Scenario	Result	Acceptance evidence
Crash	Pass	Impact, rotation, tilted rest, local alarm, and alert-due event after the 15 s countdown
Pothole	Pass	Candidate rejected and alarm GPIO remained low
Cancel	Pass	Alert cancelled; alarm GPIO low immediately and after a 16 s guard; no later alert-due event

6 Results and interpretation

Table 6 and Figure 3 report event-level results on 150 positive and 525 negative generated events. The acceleration-only baseline is sensitive but produces 289 false positives. The strict ordered rule avoids false positives but misses 111 of 150 simulated crashes. CrashGuard recovers 75 of those misses without adding a false positive in this set.

Table 6: Deterministic replay results.

Method	TP	TN	FP	FN	Sens.	Spec.	F1
Acceleration only	134	236	289	16	89.3%	45.0%	0.468
Strict ordered rule	39	525	0	111	26.0%	100.0%	0.413
CrashGuard	114	525	0	36	76.0%	100.0%	0.864

**Figure 3:** Sensitivity and specificity with Wilson 95% intervals. Intervals describe finite simulated event counts, not road uncertainty.

The CrashGuard sensitivity interval is [68.6%, 82.1%]. Its observed specificity is 100%, but the lower Wilson bound is 99.3%; zero false positives in 525 generated events does not prove that false alarms are impossible. Median confirmation delay is 2310 ms, which reflects the required two-second stable-rest interval. All 75 high-side events were detected, while 36 of 75 lower-dynamic low-side events were missed. This is the main simulated failure mode.

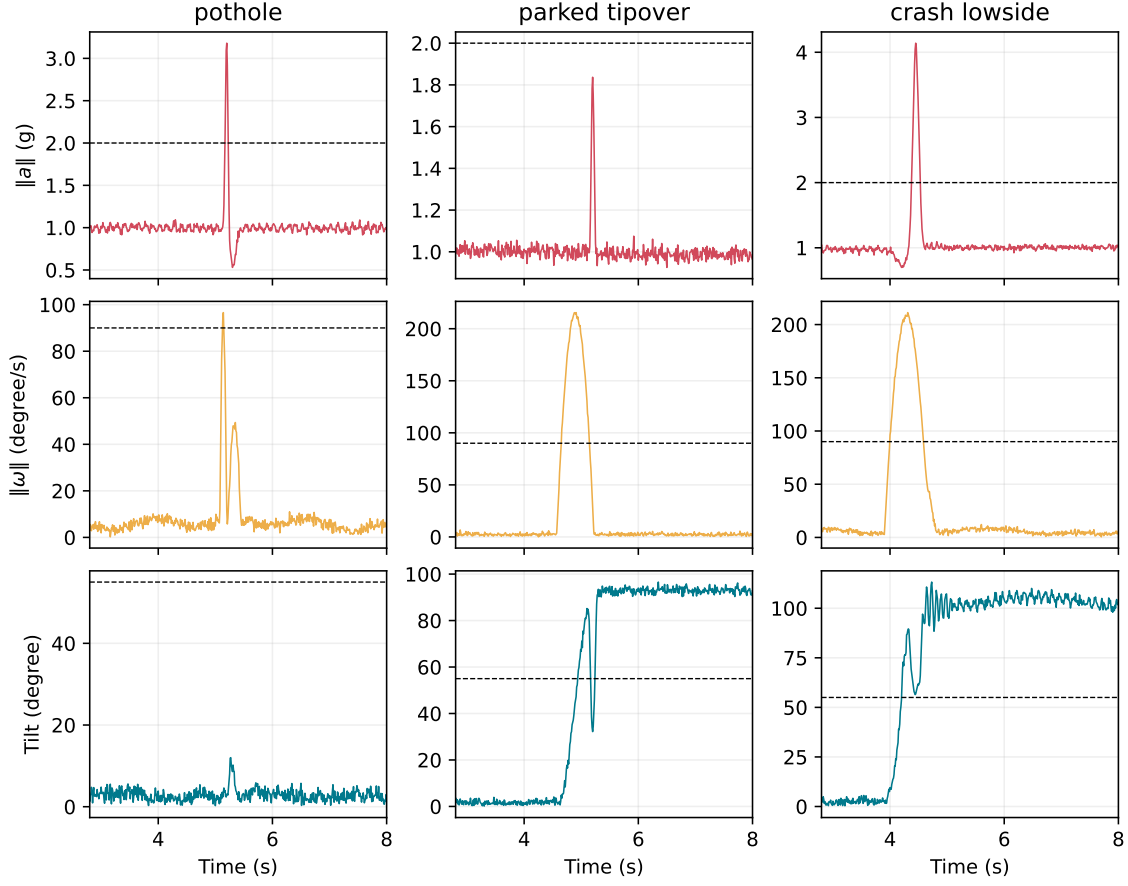


Figure 4: Example pothole, parked tip-over, and low-side crash profiles. A threshold crossing alone does not determine the output.

The independent PTW audit supports lowering the original thresholds. None of the four published fall intervals reached 4 g or 200 °/s, whereas all four passed the CrashGuard high-or-low acceleration gate and all four passed its 90 °/s rotation gate. Several non-fall manoeuvres also cross an individual CrashGuard gate, confirming why temporal correlation and final-rest evidence are necessary. The external archive does not provide a reliable gravity reference for reproducing the full state machine, so it is not reported as a complete CrashGuard accuracy test.

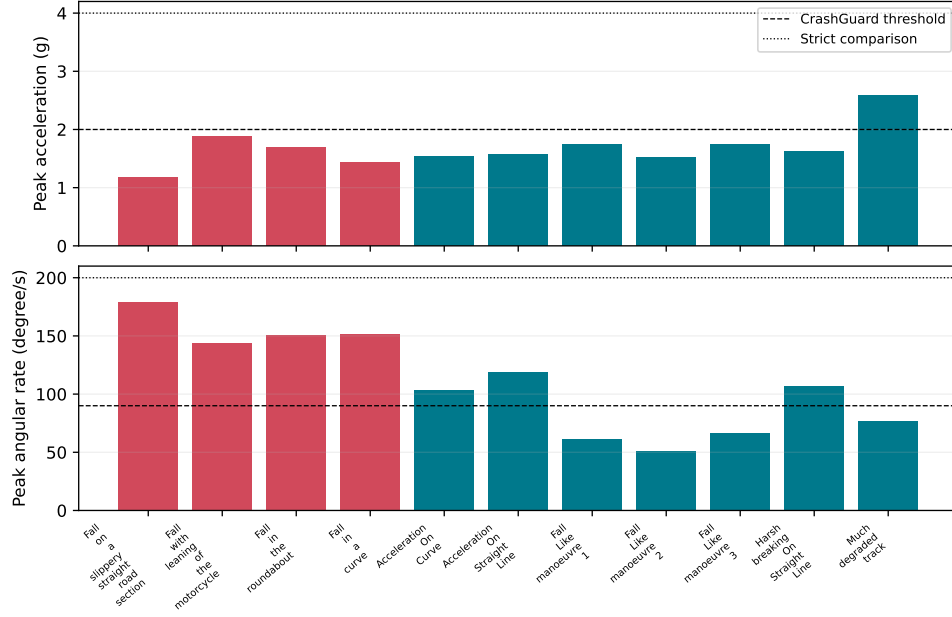


Figure 5: Peak acceleration and angular rate in four PTW falls and seven critical manoeuvre recordings after resampling to 100 Hz.

7 Limitations and safety

The following limitations define the valid scope of the results:

- The generated signals omit frame flex, mounting slip, engine harmonics, varied impact locations, weather, sensor drift, and rider- vehicle separation.
- The external PTW set contains only four controlled falls, while the human dataset has the wrong sensing domain.
- The generator and thresholds were developed in the same project; no blinded physical holdout has been evaluated.
- Event specificity is not a false-alert rate per hour or kilometre.
- Wokwi does not validate the cell, charger, boost converter, mounting, piezo loudness, Wi-Fi coverage, HTTPS delivery, or Blynk receipt.
- The prototype is not a certified automotive, medical, or emergency device. Intentional public-road crash testing would be unsafe.

Future validation should proceed in stages: bench-check voltages, sensor orientation, sounder current, sampling jitter, and charge isolation; collect unmanned stationary tip-overs and suspended-frame drops into soft containment; then log ordinary riding over potholes, braking, turns, weather, and mounting variations. Thresholds should be frozen before a held-out route, and operational results should include false alerts per hour and kilometre.

8 Conclusion

CrashGuard combines impact, rotation, tilt, rest, and rider cancellation in a compact ESP32 state machine. In deterministic simulation it improves on the strict ordered comparison and avoids the acceleration-only method’s high false-positive count. Public motorcycle recordings also show that the 4 g and 200 °/s thresholds are not portable. However, 36 simulated low-side events remain missed, so physical validation must precede stronger claims.

References

- [1] Ministry of Road Transport and Highways, Government of India, “Road accidents in india 2023,” Transport Research Wing, Tech. Rep., 2025. [Online]. Available: <https://morth.nic.in/sites/default/files/Road-Accident-in-India-2023-Publications.pdf>
- [2] J. Tian, S. Xu, P. Gerstoft, M. Sarkar, A. A. Moore, P. Mercier, and C. Paolini, “A multi-head attention transformer model for wearable in situ fall detection,” *IEEE Access*, vol. 14, pp. 85 242–85 262, 2026.
- [3] A. Boubezoul, S. Espié, B. Larnaudie, and S. Bouaziz, “A simple fall detection algorithm for powered two wheelers,” *Control Engineering Practice*, vol. 21, no. 3, pp. 286–297, 2013.
- [4] A. Boubezoul, F. Dufour, S. Bouaziz, B. Larnaudie, and S. Espié, “Dataset on powered two wheelers fall and critical events detection,” *Data in Brief*, vol. 23, p. 103828, 2019.
- [5] S. Gelmini, S. Strada, M. Tanelli, S. Savaresi, and C. D. Tommasi, “Automatic crash detection system for two-wheeled vehicles: Design and experimental validation,” *IFAC-PapersOnLine*, vol. 52, no. 5, pp. 498–503, 2019.
- [6] S. Gelmini, S. C. Strada, M. Tanelli, S. M. Savaresi, and C. D. Tommasi, “A novel crash detection algorithm for two-wheeled vehicles,” *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 1, pp. 88–99, 2021.
- [7] A. J. Mahariba, A. U. R., and G. B. Rajan, “An efficient automatic accident detection system using inertial measurement through machine learning techniques for powered two wheelers,” *Expert Systems with Applications*, vol. 192, p. 116389, 2022.
- [8] S. Lee, B. Koo, and Y. Kim, “Pre-impact fall detection for e-scooter riding using an imu: Threshold-based, supervised, and unsupervised approaches,” *Applied Sciences*, vol. 14, no. 22, p. 10443, 2024.
- [9] Blynk, “Log event: Blynk library firmware api,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.blynk.io/en/blynk-library-firmware-api/log-event>
- [10] TDK InvenSense, *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. [Online]. Available: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet.pdf>
- [11] Espressif Systems, *ESP32 Series Datasheet*, 2025. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [12] SDSU HealthLINK Center, “Falls dataset archive,” 2026, downloaded 28 August 2026. [Online]. Available: <https://osf.io/n27yz/>
- [13] Wokwi, “Diagram file format reference,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.wokwi.com/diagram-format>
- [14] —, “MPU6050 simulator and automation reference,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.wokwi.com/parts/wokwi-mpu6050>
- [15] —, “Automation scenarios,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.wokwi.com/wokwi-ci/automation-scenarios>